Prototype

Prototype Pattern

Create by cloning

DEFINITION

Create new objects by copying an existing instance (the prototype) instead of constructing from scratch.

Prototype creates objects by copying an existing instance — the prototype — rather than calling new and re-running expensive setup. The prototype acts as a live template: configure it once, then clone() to stamp out new instances, tweaking each copy as needed. New "types" become new prototype instances registered at runtime, not new classes.

WHY IT MATTERS

When construction is costly — heavy parsing, DB or IO setup — or an object's exact configuration is only known at runtime, cloning a ready instance skips the expensive initialisation and adds variants without a class explosion. The catch is copy depth: a shallow clone shares nested references, so mutating the copy can silently corrupt the original.

Vasyl Krupka · Ukraine

How it works

Prototype	Declares clone() — the contract that returns a copy of itself.
ConcretePrototype	Implements the copy, choosing shallow vs deep per field.
Client	Holds a prototype (or a registry of them) and clones instead of newing.

HOW TO APPLY

1. Find creation that is expensive, or whose configuration is fixed only at runtime.
2. Give the object a clone() that returns an independent copy.
3. Decide per field: copy nested mutable state deeply; share only truly immutable parts.
4. Keep a registry of ready prototypes and clone the one you need.

> LITMUS TEST
>
> Is new here expensive or its configuration runtime-decided, and would a copy of an existing instance do? If yes — and you can copy it safely — clone instead of build.

Smell → fix

Rebuilding a heavy object from scratch every time re-runs costly setup:

```
const d = new Doc()
d.loadTemplate() // slow parse re-run on every Doc
```

↓ CLONE A PREPARED EXEMPLAR

```
class Doc {
body = ''; tags = ['draft']
clone(): Doc { // copy self, deep where nested
const c = new Doc()
c.body = this.body; c.tags = [...this.tags]
return c
}
}
```

clone() copies a ready Doc, so the slow setup runs once on the prototype, not on every instance. Copy nested state deeply (spread / structuredClone); share only immutable parts, or copies will alias the original.

USE WHEN

- ✓ Object creation is expensive
- ✓ Many near-identical configured instances

AVOID WHEN

- ▲ Objects are cheap to build
- ▲ Deep graphs with cycles are painful to copy

— Trade-offs & pitfalls

PROS

+ Skips costly initialization
+ Runtime-configurable instances

CONS

− Deep vs shallow copy hazards
− Cloning complex graphs

COMMON PITFALLS

▲ Shallow-clone aliasing: a copy that shares nested arrays or objects with the original — mutating one corrupts the other.

▲ Cloning deep graphs with cycles, or handles (sockets, file descriptors) that can't be meaningfully copied.

▲ Using Prototype where new is already cheap — clone machinery adds complexity for no gain.

WHERE YOU SEE IT

- structuredClone() and spread copies of configured objects in JS.
- Editor "duplicate" (shape, slide, layer) that copies a fully-styled object.
- Object.create(proto) and JS's own prototype chain; game entity templates cloned per spawn.

SEE ALSO

Factory Method	an alternative creator — override new by subclass vs clone a registered instance
Abstract Factory	a factory can produce its family by cloning stored prototypes
Flyweight	opposite intent — share one instance instead of copying many
Composite	deep-cloning a Composite tree is a classic Prototype use

Interview & sources

Shallow vs deep clone — what's the trap?

A shallow copy shares nested references, so mutating the clone mutates the original. Deep-copy mutable nested state (spread, structuredClone) to keep copies independent.

Prototype vs Factory Method?

Both create without naming a concrete class at the call site — Factory Method by overriding in a subclass, Prototype by cloning a registered instance. Prototype adds or removes "types" at runtime.

When is Prototype the right call?

When new is expensive, or an object's configuration is decided at runtime and you'd rather copy a prepared exemplar than rebuild it.

How does JavaScript relate to Prototype?

JS is prototype-based at the language level: objects delegate to a prototype, and Object.create(proto) makes a new object from an exemplar — the pattern, built in.

REMEMBER

Make new objects by cloning a configured exemplar instead of building each one from scratch.

SOURCES

GoF — "Design Patterns" (1994): Prototype (creational)

Joshua Bloch — "Effective Java" (3rd ed., 2018): Item 13 — override clone judiciously (copy ctors / factories often beat it)

Vasyl Krupka · Ukraine